

Sprint 21 Structured Notes: Algorithms, Big O, and Problem-Solving Workflow

0. What Sprint 21 Is About

Sprint 21 is about learning how to think before writing code.

Before this sprint, you learned many JavaScript tools:

- values
- variables
- strings
- numbers
- booleans
- conditions
- functions
- arrays
- loops
- objects
- callbacks
- Sets and Maps
- DOM selection
- DOM updates
- events
- forms
- accessibility
- dates and media APIs
- browser storage
- debugging
- regex
- classes

Now Sprint 21 teaches a new skill:

Do not only ask, "Can I write the code?"
Ask, "Is my solution correct, clear, and efficient?"

The main goal of Sprint 21 is:

Learn how to solve coding problems in a planned way and understand how your solution grows when the input gets bigger.

By the end of Sprint 21, you should understand:

- algorithms
- correctness
- efficiency
- Big O notation
- input size `n`
- time complexity
- space complexity
- `O(1)` constant time
- `O(n)` linear time
- `O(n2)` quadratic time
- basic awareness of `O(log n)`, `O(n log n)`, `O(2n)`, and `O(n!)`
- pseudocode
- edge cases
- problem-solving workflow
- modular code
- testing
- refactoring
- how to explain a solution in simple words

1. Current Progress Before Sprint 21

Sprint 21 comes after you already know the main JavaScript language tools and many browser tools.

1.1 Core JavaScript Progress

You should already be able to work with:

Area	What You Learned
Values and variables	Store data with <code>let</code> and <code>const</code> .
Data types	Recognize strings, numbers, booleans, objects, arrays, <code>null</code> , and <code>undefined</code> .
Strings	Create, search, slice, trim, replace, and format text.
Numbers and booleans	Calculate, compare, and make decisions.
Functions	Write reusable logic with parameters and return values.
Arrays	Store ordered lists of data.
Loops	Repeat work safely.
Objects	Store structured key-value data.

Area	What You Learned
Callbacks	Pass functions into other functions.
Sets and Maps	Use special collections for unique values and key-value pairs.
Classes	Create reusable object blueprints.

1.2 Browser JavaScript Progress

You should also understand browser JavaScript basics:

Area	What You Learned
DOM selection	Find page elements with JavaScript.
DOM updates	Change text, attributes, classes, styles, and children.
Events	Respond to clicks, typing, form submission, and timers.
Accessibility	Keep visual state and accessibility state in sync.
Forms	Validate user input and control submission.
Dates and media	Use browser APIs for time, audio, video, and media features.
Storage	Save simple data with <code>localStorage</code> and <code>sessionStorage</code> .
Debugging	Find and fix errors systematically.

1.3 Why Sprint 21 Comes Here

Before Sprint 21, you learned how to build small JavaScript features.

Example:

```
function addTask(tasks, text) {
  tasks.push({
    text: text,
    done: false
  });

  return tasks;
}
```

This uses earlier skills:

- functions
- arrays

- objects
- return values

But Sprint 21 asks deeper questions:

Does this function work correctly?
What happens if text is empty?
Does it change the original array?
How much work does it do?
How much memory does it use?
Can I explain the solution clearly?

Simple summary:

Earlier sprints taught you how to write JavaScript.
Sprint 21 teaches you how to reason about JavaScript solutions.

2. The Main Shift in Sprint 21

Before Sprint 21, you often asked:

Does my code run?

In Sprint 21, you also ask:

Does my code solve the correct problem?
Does it handle edge cases?
Is it clear?
Is it efficient enough?
How does it behave when the input grows?

This is the difference between writing code and solving problems.

Beginner Mental Model

Problem
↓
Understand the input and output
↓
Plan the steps
↓
Write pseudocode

```
↓  
Write JavaScript  
↓  
Test normal cases  
↓  
Test edge cases  
↓  
Analyze time and space  
↓  
Refactor if needed
```

3. What Is an Algorithm?

An algorithm is a clear set of steps that solves a problem.

Simple meaning:

An algorithm is a recipe for solving a coding problem.

A cooking recipe tells you:

1. Add flour.
2. Add water.
3. Mix.
4. Bake.

An algorithm tells the computer:

1. Read the input.
2. Check each value.
3. Save the result.
4. Return the answer.

3.1 Algorithm Example: Reverse a String

Problem:

Reverse a string.

Input:

"hello"

Output:

"olleh"

Algorithm idea:

Take the original string.
Break it into characters.
Reverse the order.
Join the characters back into a string.
Return the result.

JavaScript:

```
function reverseString(str) {  
  return str.split("").reverse().join("");  
}  
  
console.log(reverseString("hello")); // "olleh"
```

3.2 What Makes a Good Algorithm?

A good algorithm should be:

Quality	Meaning
Clear	The steps are understandable.
Precise	Each step says exactly what to do.
Finite	It eventually stops.
Correct	It gives the expected result.
Efficient enough	It does not waste too much time or memory.

3.3 Bad Algorithm Example

This is not a useful algorithm:

Keep trying until it works.

Why is it weak?

Because it is not precise.

A better algorithm:

```
Loop through every item.  
Compare the item to the target.  
If the item matches the target, return true.  
If the loop finishes with no match, return false.
```

4. Correctness

Correctness means the algorithm gives the right answer.

Simple meaning:

A correct algorithm returns the expected output for the given input.

4.1 Correctness Example

Problem:

Return the first item in an array.

Code:

```
function getFirstItem(arr) {  
  return arr[0];  
}  
  
console.log(getFirstItem(["a", "b", "c"])); // "a"
```

This works for a normal array.

But what about an empty array?

```
console.log(getFirstItem([])); // undefined
```

That may or may not be acceptable. You must decide based on the problem.

A safer version:

```
function getFirstItem(arr) {  
  if (arr.length === 0) {  
    return null;  
  }  
  
  return arr[0];  
}  
  
console.log(getFirstItem(["a", "b"])); // "a"  
console.log(getFirstItem([]));         // null
```

4.2 Correctness Questions

Before calling your solution correct, ask:

```
Does it work for normal input?  
Does it work for empty input?  
Does it work for one item?  
Does it work for repeated values?  
Does it work for negative numbers, if numbers are involved?  
Does it return the correct type?  
Does it stop at the right time?
```

5. Edge Cases

An edge case is an unusual but valid input.

Simple meaning:

Edge cases are inputs that are easy to forget but can break your code.

5.1 Common Edge Cases

Problem Type	Common Edge Cases
Array problems	Empty array, one item, duplicate items, missing item.
String problems	Empty string, uppercase/lowercase, spaces, special characters.
Number problems	Zero, negative numbers, decimals, very large numbers.
Object problems	Missing property, <code>null</code> , nested missing data.
Search problems	Target exists, target does not exist, target appears more than once.

5.2 Edge Case Example

Problem:

Find the largest number in an array.

Normal input:

[4, 10, 2, 99, 8]

Expected output:

99

Edge cases:

```
[ ]           // empty array
[7]          // one item
[-5, -2, -10] // negative numbers
[10, 10, 5]  // duplicate largest values
```

A good solution should decide what to do for each case.

6. Efficiency

Efficiency means how much work and memory your solution uses.

There are two main types:

Time complexity -> how much work the algorithm does
Space complexity -> how much extra memory the algorithm uses

Example:

```
function printItems(items) {  
  for (const item of items) {  
    console.log(item);  
  }  
}
```

If the array has 5 items, the loop runs 5 times.

If the array has 1,000 items, the loop runs 1,000 times.

The amount of work grows with the input size.

7. What Is Big O Notation?

Big O notation describes how an algorithm grows when the input gets bigger.

Simple meaning:

Big O tells us how the work or memory changes as the input size grows.

Big O does not mainly ask:

How many milliseconds did this take on my laptop?

It asks:

If the input gets bigger, how does the work grow?

7.1 What Is n ?

In Big O, n usually means the input size.

Example:

```
const arr = [10, 20, 30, 40, 50];
```

Here:

```
n = 5
```

If the array has 1,000 items:

```
n = 1000
```

If the string has 20 characters:

```
n = 20
```

7.2 Big O Ignores Small Details

Big O focuses on growth.

It usually ignores:

- exact milliseconds
- computer speed
- browser differences
- small constant differences
- lower-order terms

Example:

```
function printTwice(arr) {  
  for (const item of arr) {  
    console.log(item);  
  }  
  
  for (const item of arr) {  
    console.log(item);  
  }  
}
```

This loops through the array twice.

Exact work:

2n

Big O:

$O(n)$

Why?

Because Big O ignores the constant `2` and focuses on the growth pattern.

8. Time Complexity

Time complexity describes how much work an algorithm does.

Simple meaning:

Time complexity answers: “How many steps does this solution need as input grows?”

9. `O(1)`: Constant Time

`O(1)` means the work stays about the same no matter how large the input is.

Simple meaning:

Constant time means the algorithm does not loop through the whole input.

9.1 Example: Check Whether an Array Has a First Item

```
function hasFirstItem(arr) {  
  return arr.length > 0;  
}  
  
console.log(hasFirstItem([10, 20, 30])); // true  
console.log(hasFirstItem([]));           // false
```

This function checks `arr.length` once.

It does not matter if the array has:

```
3 items  
300 items  
3,000,000 items
```

The function still performs one main check.

Time complexity:

```
O(1)
```

Space complexity:

```
O(1)
```

9.2 More $O(1)$ Examples

```
function getFirstItem(arr) {  
  return arr[0];  
}
```

```
function getLastItem(arr) {  
  return arr[arr.length - 1];  
}
```

```
function isAdult(age) {  
  return age >= 18;  
}
```

Each one does a fixed amount of work.

10. $O(n)$: Linear Time

$O(n)$ means the work grows directly with the input size.

Simple meaning:

Linear time usually means one full pass through the input.

10.1 Example: Print Every Item

```
function printAll(arr) {  
  for (const item of arr) {  
    console.log(item);  
  }  
}
```

If the array has 3 items, the loop runs 3 times.

If the array has 1,000 items, the loop runs 1,000 times.

Time complexity:

$O(n)$

Space complexity:

$O(1)$

Why space is $O(1)$?

Because we are not creating a new array that grows with the input.

10.2 More $O(n)$ Examples

```
function countItems(arr) {  
  let count = 0;  
  
  for (const item of arr) {  
    count++;  
  }  
  
  return count;  
}
```

```
function containsTarget(arr, target) {  
  for (const item of arr) {  
    if (item === target) {  
      return true;  
    }  
  }  
}
```

```
    return false;
}
```

```
function sumNumbers(numbers) {
  let total = 0;

  for (const number of numbers) {
    total += number;
  }

  return total;
}
```

Each function loops through the input at most once.

11. $O(n^2)$: Quadratic Time

$O(n^2)$ means the work grows much faster than $O(n)$.

It often happens when you have nested loops over the same growing input.

Simple meaning:

Quadratic time often means comparing many items with many other items.

11.1 Example: Print All Pairs

```
function printPairs(arr) {
  for (const a of arr) {
    for (const b of arr) {
      console.log(a, b);
    }
  }
}
```

If the array has 3 items:

$3 \times 3 = 9$ pairs

If the array has 10 items:

$$10 \times 10 = 100 \text{ pairs}$$

If the array has 1,000 items:

$$1,000 \times 1,000 = 1,000,000 \text{ pairs}$$

Time complexity:

$$O(n^2)$$

Space complexity:

$$O(1)$$

11.2 Important Correction About Nested Loops

Nested loops are not always $O(n^2)$.

But nested loops over the same growing input are often $O(n^2)$.

Example of $O(n^2)$:

```
for (const first of arr) {  
  for (const second of arr) {  
    console.log(first, second);  
  }  
}
```

The outer loop depends on `arr`.

The inner loop also depends on `arr`.

So the work grows like:

$$n \times n$$

That becomes:

$O(n^2)$

12. Common Big O Types

You do not need to master every Big O type immediately, but you should recognize the names.

Big O	Name	Beginner Meaning	Example
$O(1)$	Constant	Same amount of work	Get first item.
$O(\log n)$	Logarithmic	Cut the problem down repeatedly	Binary search.
$O(n)$	Linear	Loop once through input	Print all items.
$O(n \log n)$	Linearithmic	Common efficient sorting range	Merge sort.
$O(n^2)$	Quadratic	Nested growth	Compare every pair.
$O(2^n)$	Exponential	Doubles rapidly	Some naive recursive solutions.
$O(n!)$	Factorial	Extremely explosive growth	All permutations.

12.1 Main Big O Types for Sprint 21

For Sprint 21, focus mostly on:

$O(1)$
 $O(n)$
 $O(n^2)$

Those are the most important beginner categories.

13. Space Complexity

Space complexity describes how much extra memory an algorithm uses.

Simple meaning:

Space complexity answers: "How much extra storage does this solution need?"

Extra memory can include:

- new arrays

- new objects
 - new Sets
 - new Maps
 - temporary lists
 - recursive call stack memory
-

14. $O(1)$ Space

$O(1)$ space means the algorithm uses about the same amount of extra memory no matter how large the input is.

14.1 Example: Sum Numbers

```
function getTotal(arr) {  
  let total = 0;  
  
  for (const number of arr) {  
    total += number;  
  }  
  
  return total;  
}
```

This function uses one extra variable:

```
let total = 0;
```

Even if the array has 10 items or 10,000 items, the function mainly keeps one extra variable.

Time complexity:

```
 $O(n)$ 
```

Space complexity:

```
 $O(1)$ 
```

15. $O(n)$ Space

$O(n)$ space means the extra memory grows with the input size.

15.1 Example: Double Numbers Into a New Array

```
function doubleNumbers(arr) {  
  const result = [];  
  
  for (const number of arr) {  
    result.push(number * 2);  
  }  
  
  return result;  
}
```

If the input has 5 numbers, `result` gets 5 numbers.

If the input has 1,000 numbers, `result` gets 1,000 numbers.

Time complexity:

$O(n)$

Space complexity:

$O(n)$

Why space is $O(n)$?

Because the new array grows as the input grows.

16. Time Complexity vs Space Complexity

Time and space are different.

A function can be:

$O(n)$ time and $O(1)$ space

or:

$O(n)$ time and $O(n)$ space

16.1 Example: $O(n)$ Time and $O(1)$ Space

```
function countPositive(numbers) {  
  let count = 0;  
  
  for (const number of numbers) {  
    if (number > 0) {  
      count++;  
    }  
  }  
  
  return count;  
}
```

This loops once, but only stores `count`.

Time: $O(n)$
Space: $O(1)$

16.2 Example: $O(n)$ Time and $O(n)$ Space

```
function getPositiveNumbers(numbers) {  
  const positives = [];  
  
  for (const number of numbers) {  
    if (number > 0) {  
      positives.push(number);  
    }  
  }  
  
  return positives;  
}
```

This loops once and creates a new array.

Time: $O(n)$
Space: $O(n)$

17. Pseudocode

Pseudocode means writing the plan in plain language before writing real code.

Simple meaning:

Pseudocode is a rough draft of your logic.

It is not JavaScript.

It is not meant to run.

It helps you think.

17.1 Pseudocode Example

Problem:

Find the largest number in an array.

Pseudocode:

```
IF the array is empty, return null.  
SET largest to the first item.  
FOR each number in the array:  
    IF number is greater than largest:  
        SET largest to number.  
RETURN largest.
```

This helps before writing JavaScript because the logic is clear.

17.2 Why Pseudocode Helps

Pseudocode helps you:

- slow down
- understand the problem
- avoid guessing
- find edge cases earlier
- explain your idea
- write cleaner JavaScript

Beginner rule:

If you cannot explain the steps in plain words, you are probably not ready to code the solution yet.

18. Sprint 21 Problem-Solving Workflow

Use this workflow for algorithm problems.

1. Read the problem.
2. Identify the input.
3. Identify the output.
4. Understand the transformation.
5. List edge cases.
6. Write pseudocode.
7. Choose useful data structures.
8. Write the JavaScript.
9. Test normal cases.
10. Test edge cases.
11. Analyze time complexity.
12. Analyze space complexity.
13. Refactor if needed.

19. Problem-Solving Template

Use this template before coding:

```
Problem:
Input:
Output:
Transformation:
Constraints:
Edge cases:
Pseudocode:
Code:
Tests:
Time complexity:
Space complexity:
Refactor notes:
```

19.1 What Each Part Means

Part	Meaning
Problem	What are you trying to solve?
Input	What data does the function receive?
Output	What should the function return?
Transformation	How does input become output?
Constraints	Any limits or rules?
Edge cases	What unusual inputs must be handled?
Pseudocode	Plain-language plan.
Code	JavaScript solution.
Tests	Examples that prove the code works.
Time complexity	How work grows.
Space complexity	How memory grows.
Refactor notes	How to improve clarity or structure.

20. Full Example: Find the Largest Number

Problem:

```
Write a function that returns the largest number in an array.
```

20.1 Input

```
An array of numbers.
```

Example:

```
[4, 10, 2, 99, 8]
```

20.2 Output

```
The largest number.
```

Example:

99

20.3 Transformation

Look at every number.
Keep track of the largest number seen so far.
Return the largest number after checking all numbers.

20.4 Edge Cases

Empty array
One number
Negative numbers
Duplicate largest values

20.5 Pseudocode

IF the array is empty, return null.
SET largest to the first item.
FOR each number in the array:
 IF number is greater than largest:
 SET largest to number.
RETURN largest.

20.6 Code

```
function findLargest(numbers) {  
  if (numbers.length === 0) {  
    return null;  
  }  
  
  let largest = numbers[0];  
  
  for (const number of numbers) {  
    if (number > largest) {  
      largest = number;  
    }  
  }  
}
```



```
    return largest;
}
```

20.7 Tests

```
console.log(findLargest([4, 10, 2, 99, 8])); // 99
console.log(findLargest([-5, -2, -10]));    // -2
console.log(findLargest([7]));              // 7
console.log(findLargest([10, 10, 5]));      // 10
console.log(findLargest([]));               // null
```

20.8 Complexity

The function loops through the array once.

Time complexity: $O(n)$

The function only stores one extra variable, `largest`.

Space complexity: $O(1)$

21. Mini Lesson: Classifying Functions

Classifying means looking at a function and deciding its time and space complexity.

21.1 Function A: Get Last Item

```
function getLastItem(arr) {
    return arr[arr.length - 1];
}
```

This does one direct access.

Time: $O(1)$
Space: $O(1)$

Why?

Because it does not loop through the array.

21.2 Function B: Count Items

```
function countItems(arr) {  
  let count = 0;  
  
  for (const item of arr) {  
    count++;  
  }  
  
  return count;  
}
```

This loops once.

```
Time: O(n)  
Space: O(1)
```

Why?

Because the loop grows with the array size, but memory stays almost constant.

21.3 Function C: Create a Copy

```
function createCopy(arr) {  
  const copy = [];  
  
  for (const item of arr) {  
    copy.push(item);  
  }  
  
  return copy;  
}
```

This loops once and creates a new array.

```
Time: O(n)  
Space: O(n)
```

Why?

Because `copy` grows as the input grows.

21.4 Function D: Print All Pairs

```
function printAllPairs(arr) {  
  for (const first of arr) {  
    for (const second of arr) {  
      console.log(first, second);  
    }  
  }  
}
```

This uses nested loops over the same array.

```
Time:  $O(n^2)$   
Space:  $O(1)$ 
```

Why?

Because every item is paired with every item.

22. Full Beginner Practice: Count Vowels

Problem:

```
Write a function that counts vowels in a string.
```

Vowels:

```
a, e, i, o, u
```

22.1 Input

```
A string.
```

Example:

"hello"

22.2 Output

A number .

Example:

2

Because `hello` has:

e
o

22.3 Transformation

Look at each character.
Convert it to lowercase.
Check whether it is a vowel.
If it is a vowel, increase the count.
Return the final count.

22.4 Edge Cases

Empty string
String with no vowels
Uppercase vowels
Mixed-case string
Spaces and punctuation

22.5 Pseudocode

```
SET count to 0.  
SET vowels to "aeiou".  
FOR each character in the string:  
    Convert character to lowercase.  
    IF vowels includes the character:
```

```
Add 1 to count.  
RETURN count.
```

22.6 Code

```
function countVowels(text) {  
  let count = 0;  
  const vowels = "aeiou";  
  
  for (const char of text.toLowerCase()) {  
    if (vowels.includes(char)) {  
      count++;  
    }  
  }  
  
  return count;  
}
```

22.7 Tests

```
console.log(countVowels("hello")); // 2  
console.log(countVowels("JavaScript")); // 3  
console.log(countVowels("xyz")); // 0  
console.log(countVowels("")); // 0  
console.log(countVowels("AEIOU")); // 5
```

22.8 Complexity

Let `n` be the number of characters in `text`.

The function checks each character once.

Time complexity: $O(n)$

The function only stores a few variables:

```
count  
vowels  
char
```

It does not create a new array that grows with the input.

Space complexity: $O(1)$

23. Modular Code

Modular code means splitting a solution into smaller pieces.

Simple meaning:

Modular code is code organized into small functions that each have one clear job.

23.1 Non-Modular Example

```
function analyzeText(text) {  
  let count = 0;  
  const vowels = "aeiou";  
  
  for (const char of text.toLowerCase()) {  
    if (vowels.includes(char)) {  
      count++;  
    }  
  }  
  
  console.log(`The text has ${count} vowels.`);  
}
```

This function does two jobs:

1. Count vowels.
2. Print a message.

23.2 More Modular Version

```
function countVowels(text) {  
  let count = 0;  
  const vowels = "aeiou";  
  
  for (const char of text.toLowerCase()) {  
    if (vowels.includes(char)) {  
      count++;  
    }  
  }  
}
```

```

    return count;
}

function formatVowelMessage(text) {
    const count = countVowels(text);
    return `The text has ${count} vowels.`;
}

console.log(formatVowelMessage("JavaScript"));

```

Now each function has a clearer job.

Function	Job
<code>countVowels()</code>	Calculate the number of vowels.
<code>formatVowelMessage()</code>	Create a message.

Beginner rule:

A function is easier to test when it does one main thing.

24. Testing

Testing means checking whether your code works as expected.

Simple meaning:

Testing proves your function works for normal cases and edge cases.

24.1 Basic Manual Tests

```

console.log(findLargest([4, 10, 2])); // 10
console.log(findLargest([]));         // null

```

This is a simple manual test.

You run the function and compare the result with the expected output.

24.2 Better Test Thinking

For each function, test:

Normal input
Empty input
One-item input
Input with repeated values
Input with missing target
Input with unusual but valid values

24.3 Example Test List for `containsTarget`

Problem:

Return true if an array contains a target value, otherwise return false.

Tests:

```
console.log(containsTarget([1, 2, 3], 2)); // true
console.log(containsTarget([1, 2, 3], 5)); // false
console.log(containsTarget([], 1));        // false
console.log(containsTarget([7], 7));       // true
console.log(containsTarget([7], 2));       // false
```

25. Refactoring

Refactoring means improving code without changing what it does.

Simple meaning:

Refactoring makes code cleaner while keeping the same behavior.

You refactor to improve:

- readability
- naming
- structure
- repeated logic
- testability
- maintainability

25.1 Before Refactoring

```
function f(a) {  
  let x = 0;  
  
  for (const y of a) {  
    x += y;  
  }  
  
  return x;  
}
```

This works, but the names are unclear.

25.2 After Refactoring

```
function sumNumbers(numbers) {  
  let total = 0;  
  
  for (const number of numbers) {  
    total += number;  
  }  
  
  return total;  
}
```

The behavior is the same, but the code is easier to read.

25.3 Refactoring Rule

Do not refactor randomly.

Use this order:

1. Make it work.
2. Test it.
3. Make it clearer.
4. Test it again.

26. Practice Problem: Contains Target

Problem:

Return true if an array contains a target value, otherwise return false.

26.1 Input

An array and a target value.

Example:

```
["html", "css", "javascript"], "css"
```

26.2 Output

A boolean: true or false.

Example:

```
true
```

26.3 Transformation

Check each item.
If one item matches the target, return true.
If no item matches after checking everything, return false.

26.4 Edge Cases

Empty array
Target is first item
Target is last item
Target appears more than once
Target does not exist
Array has one item

26.5 Pseudocode

```
FOR each item in the array:  
  IF item equals target:
```

```
    RETURN true.  
    RETURN false.
```

26.6 Code

```
function containsTarget(arr, target) {  
  for (const item of arr) {  
    if (item === target) {  
      return true;  
    }  
  }  
  
  return false;  
}
```

26.7 Tests

```
console.log(containsTarget(["html", "css", "javascript"], "css")); // true  
console.log(containsTarget(["html", "css"], "python"));           // false  
console.log(containsTarget([], "css"));                           // false  
console.log(containsTarget(["js"], "js"));                        // true  
console.log(containsTarget(["js"], "css"));                       // false
```

26.8 Complexity

Worst case: the target is not found, so the function checks every item.

Time complexity: $O(n)$

The function does not create a growing data structure.

Space complexity: $O(1)$

27. Common Beginner Mistakes in Sprint 21

Mistake 1: Coding Before Understanding the Problem

Weak approach:

Start typing immediately.

Better approach:

Understand input, output, edge cases, and steps first.

Mistake 2: Ignoring Edge Cases

Weak approach:

```
function getFirstItem(arr) {  
  return arr[0];  
}
```

This may be fine, but it ignores the empty array case.

Better approach if the problem needs safety:

```
function getFirstItem(arr) {  
  if (arr.length === 0) {  
    return null;  
  }  
  
  return arr[0];  
}
```

Mistake 3: Confusing `return` and `console.log()`

Weak version:

```
function add(a, b) {  
  console.log(a + b);  
}  
  
const result = add(2, 3);  
console.log(result); // undefined
```

Better version:

```
function add(a, b) {  
  return a + b;  
}  
  
const result = add(2, 3);  
console.log(result); // 5
```

For algorithm problems, functions usually need to `return` the answer.

Mistake 4: Saying Every Loop Is Bad

Loops are not bad.

A single loop is often normal and efficient enough.

```
function sumNumbers(numbers) {  
  let total = 0;  
  
  for (const number of numbers) {  
    total += number;  
  }  
  
  return total;  
}
```

This is `O(n)`, which is usually fine for many beginner problems.

Mistake 5: Saying Nested Loops Are Always Wrong

Nested loops are not automatically wrong.

Sometimes the problem requires comparing pairs.

But you should recognize that nested loops over the same input often create `O(n2)` time.

Mistake 6: Forgetting Space Complexity

A solution may be fast enough but use extra memory.

Example:

```
function copyItems(items) {  
  return [...items];  
}
```

This creates a new array.

Space: $O(n)$

Mistake 7: Not Testing Empty Input

Always ask:

What happens if the array is empty?
What happens if the string is empty?

28. Sprint 21 Summary

Sprint 21 teaches you how to think like a problem solver.

You learned that:

An algorithm is a clear, finite set of steps.
Correctness means the algorithm returns the expected result.
Edge cases are unusual but valid inputs.
Big O describes growth as input size increases.
Time complexity measures work.
Space complexity measures extra memory.
 $O(1)$ means constant time.
 $O(n)$ means linear time.
 $O(n^2)$ means quadratic time.
Pseudocode helps you plan before coding.
Testing checks normal and edge cases.
Refactoring improves code without changing behavior.

29. Minimum Passing Standard for Sprint 21

You are ready to move past Sprint 21 when you can look at a beginner function and explain:

1. What input it receives.
2. What output it returns.
3. What transformation it performs.
4. What edge cases matter.
5. Whether the time complexity is $O(1)$, $O(n)$, or $O(n^2)$.
6. Whether the space complexity is $O(1)$ or $O(n)$.
7. Whether the code is clear enough or needs refactoring.
8. What tests prove it works.

30. Quick Review Questions

Use these to check your understanding.

Question 1

What is an algorithm?

Answer:

A clear set of steps that solves a problem.

Question 2

What does Big O describe?

Answer:

How work or memory grows as input size grows.

Question 3

What does `n` usually mean?

Answer:

The size of the input.

Question 4

What is the time complexity of this function?

```
function getFirst(arr) {  
  return arr[0];  
}
```

Answer:

$O(1)$

Question 5

What is the time complexity of this function?

```
function printItems(arr) {  
  for (const item of arr) {  
    console.log(item);  
  }  
}
```

Answer:

$O(n)$

Question 6

What is the time complexity of this function?

```
function printPairs(arr) {  
  for (const a of arr) {  
    for (const b of arr) {  
      console.log(a, b);  
    }  
  }  
}
```

Answer:

$O(n^2)$

Question 7

What is space complexity?

Answer:

How much extra memory an algorithm uses.

Question 8

Why use pseudocode?

Answer:

To plan the solution in plain words before writing JavaScript.

31. Final Sprint 21 Mental Model

When solving a problem, think in this order:

Understand first.
Plan second.
Code third.
Test fourth.
Analyze fifth.
Refactor sixth.

Do not measure progress by how fast you start typing.

Measure progress by how clearly you can explain the solution.